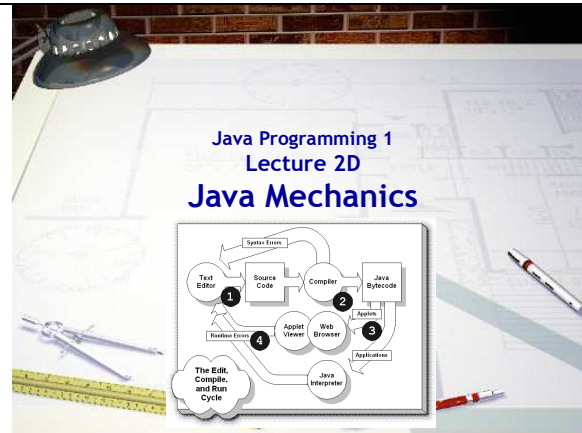




Slide 1
Java Programming 1
Lecture 2D
Java Mechanics
 Duration: 00:01:06
 Advance mode: Auto

Notes:

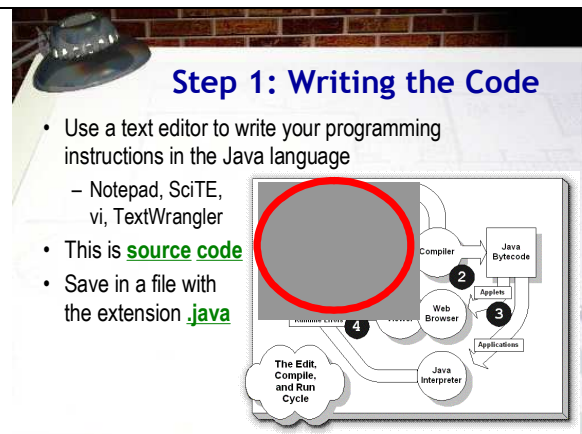
To create your own Java programs, you follow a mechanical process, a well-defined set of steps called the edit-compile-run cycle that looks like this:

The edit-compile-run cycle.

1. Create your source code
2. Compile the code to produce object, or bytecode
3. If syntax errors occur, return to your editor to fix them
4. Run your program using appletviewer or the Java interpreter
5. If runtime errors occur, return to your editor to fix them

Learning this process is not the same as learning how to program; you must master the process--that is necessary--but mastering the process doesn't mean you'll write correct and elegant programs.

Shortly we'll take a closer look at the steps in the process. As we do that, I'll ask you to complete some exercises and paste a screenshot into your in-class homework document. Check the Web page for this week for more instructions on setting up and submitting your in-class homework document.



Slide 2
Step 1: Writing the Code
 Duration: 00:02:02
 Advance mode: Auto

- Use a text editor to write your programming instructions in the Java language
 - Notepad, SciTE, vi, TextWrangler
- This is **source code**
- Save in a file with the extension **.java**

Notes:

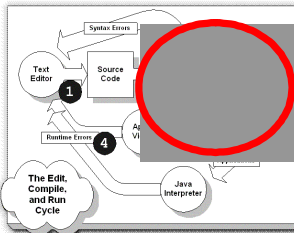
When you write a program in a high-level language, you write your instructions--in the form of programming language statements--using a text editor. A text editor is a little bit like a word processor--say Microsoft Word or WordPad--but without all of the formatting options. While this might not seem like an important distinction, it really is. When you create a document with a word processor, the word processor saves special formatting characters in your file, along with the content of your document. Your Java compiler will have a big problem if you attempt to feed it code written using a word processor instead of a text editor, because it doesn't know how to make sense of the special formatting codes. Often, you can tell your word processor to save only the text, and no formatting information, but it's easy to forget to do that.

Most operating systems come with a built-in text editor. In Microsoft Windows, that's the Notepad program, which is also available from the Accessories menu. Notepad is a fairly rudimentary editor, that has a tendency to rename its files, appending a superfluous ".txt" to whatever filename you specify. To avoid this, put double-quotes around your filename whenever you save a file using Notepad.

For most of this class, I'll assume that you're using the Scintilla Text Editor, or SciTE. This is my favorite editor on Windows and Linux. SciTE is an open-source, free text editor that includes syntax highlighting (for Java, HTML, and a

variety of other languages) and the ability to run your compiler from within the text editor. If you're programming on the Mac you can use the free programmer's editor developed by BareBones software, called TextWrangler 2.0.

The document that contains your java instructions is called source code and all Java source code is contained in files that use the extension .java.



Step 2: Compiling

- Use a Java **compiler** to translate your source code into executable machine code
 - javac, Eclipse
- This is **bytecode**
- Produces a file with the extension **.class**
- If **syntax errors** occur, return to text editor

Slide 3
Step 2: Compiling
Duration: 00:01:22
Advance mode: Auto

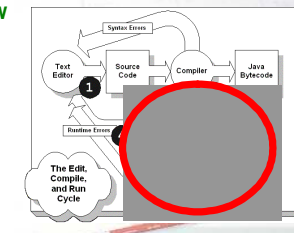
Notes:

Once you've written your program, you compile it by using a software program called, (surprise!), a compiler. The compiler turns your source code into machine language and produces another document called object code. The java compiler that comes that comes with the JDK is called javac, but others are free to write their own compilers. The GCC project has its own java compiler (gcj) while the Eclipse IDE also uses its own compiler.

The machine code or object code produced by the Java compiler is called bytecode which is the native code designed to run on the Java Virtual Machine (rather than being machine code for a particular brand of hardware CPU.) This machine code is stored in a file with the same basename as your source code file, but with the extension .class rather than .java.

If your program fails to compile, (that is, if you have "grammatical" or syntax errors in your source code), then you must re-edit the code until it compiles correctly. The compiler cannot produced a class file unless your source code is syntactically correct.

Once a program compiles without errors, it is syntactically correct. It still may contain runtime or logic errors, however.



Step 3: Run your Program

- Use a Java **interpreter** or JIT compiler to translate the bytecode into native machine code
- Applications: **java, javaw**
- Applets: HTML
 - Web browser, appletviewer
- If **runtime errors** occur, return to text editor

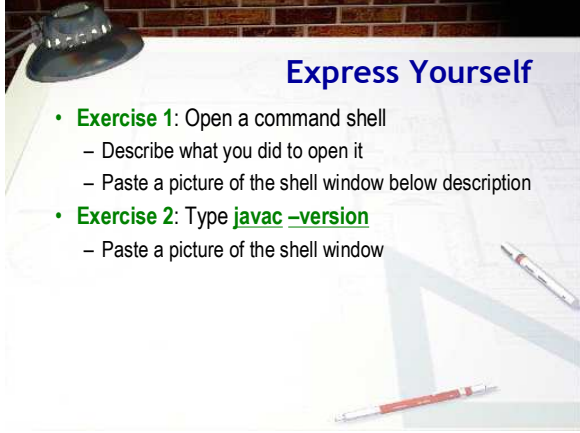
Slide 4
Step 3: Run your Program
Duration: 00:01:59
Advance mode: Auto

Notes:

To run your program, you'll start a Java Virtual Machine to load your class file, which will then use a Java interpreter or JIT compiler to convert the bytecode into the native machine code that your CPU can understand. In the JDK, you launch the JVM by using the program named java.exe. If you have a GUI program, you can also start the JVM without showing a console window by using the program javaw.exe.

The process for running an applet is a little different. Since applets are "hosted" inside Web pages, you first need to create an HTML page containing an applet tag. Then, you open the HTML page in your Web browser or the appletviewer program. This starts up the JVM and runs your program. Often your IDE (such as BlueJ or Eclipse) will automatically create the HTML page and launch the appletviewer for you.

	All high-level languages require a significant amount of additional machine-language code--in addition to the code you've written--before your program will actually run. How this additional code is added to your program varies from language to language and from system to system. Generically, this is called linking. In Java, this linking happens behind the scenes when you run your program, and you don't really have to worry about it. When you run your program, the first thing you'll want to do is to verify that it runs as you expect it. This process is called testing. It is not uncommon to find that your program has some logic errors that can only be discovered when the program runs. These errors may even make your program "crash". These kinds of errors are called runtime errors. To fix a runtime error, you go back to the very first step--your source code--and make changes there, repeating the edit-compile-run cycle until the program runs correctly.
 Slide 5  The Tools We'll Use Duration: 00:03:33 Advance mode: Auto	Notes: Let's start by taking a quick look at the tools we'll use in this class. This class is an introductory transfer Computer Science class that focuses on learning programming concepts, using the Java programming language. Even though we will write some Java applets in this class, we really won't be spending any time learning about Web development or Web page design. We also won't be using any kind of Rapid Application Development (or RAD) tool that allows you to do "drag and drop" programming. The entire class will concentrate on writing programming code. The first tool you're going to learn to use is the Java Development Kit or JDK. (This is sometimes also called the J2SE SDK or Software Development Kit. Both of those are the same thing.) The JDK can be freely downloaded from Sun Microsystems and you must install the JDK before you can use any of the other Java Development tools such as BlueJ or Eclipse. To download and install the JDK, follow the instructions in the separate online lecture: Installing the JDK. Of course, if you like, you can do all of your work in the OCC Computing Center, and then you won't have to install anything. Even if you don't install the JDK, though, you'll need to know about the different versions of Java. There are five major versions of the Java platform or J2SE (which stands for Java 2 Standard Edition): J2SE 1.3, J2SE 1.4, J2SE 1.5, and J2SE 1.6. Each of these revisions represents a major change in the class libraries and sometimes the underlying Java language. And, just to further confuse things, with the release of J2SE 1.5 the official name was changed to Java 5. For this class, I'll assume you are using the latest revision of the 1.6 branch of the Java 2 platform, (that is, Java 6), unless you're working on the Mac, where you should use Java 5. We'll be using Java 6 in the labs. The JDK consists of tools that you'll run from the command-

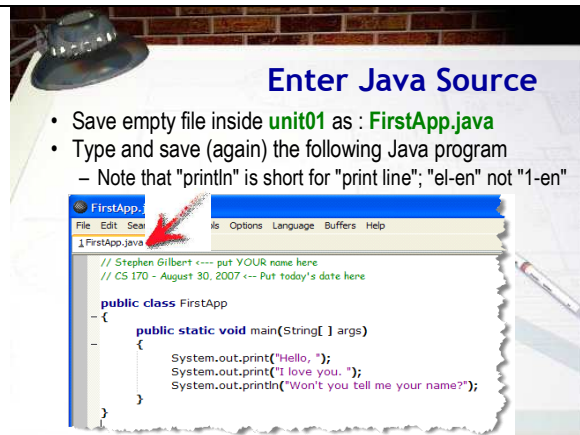
	line. These include: • <code>javac</code>: the Java compiler. <code>Javac</code> will convert your Java source code files into bytecode that can be run on the JVM. • <code>*java</code>: the program that launches the Java Virtual Machine. You'll use <code>java</code> to run your programs after you've compiled them. • <code>*appletviewer</code>: the program you'll use to test your applets, so you don't need to close your Web browser between runs. While this is all you need to compile and run your Java programs, the JDK doesn't come with any tools that help you write your code. For that, you're going to use a plain text editor. In this class we'll use a small, open-source programmer's editor named SciTE, or the Scintilla Text Editor. In addition to these "bare metal" tools, we'll also be using two Integrated Development Environments (or IDEs). During the first part of the semester, we'll use the BlueJ IDE, which is an interactive, object-based IDE developed especially for education. In the second half of the semester, once you're comfortable with Java itself, we'll switch over to a professional-level IDE known as Eclipse. Eclipse was originally created by IBM's tools division and marketed as Visual Age for Java. Today, it's an independent open-source project that you can use for free.
 Express Yourself • Exercise 1: Open a command shell – Describe what you did to open it – Paste a picture of the shell window below description • Exercise 2: Type <code>javac -version</code> – Paste a picture of the shell window Slide 6 Express Yourself Duration: 00:00:52 Advance mode: Auto	Notes: Let's start by making sure that you have the JDK installed correctly. For Exercise 1, open a command shell window. In your exercise document describe what you did to open it in your own words, and then paste a picture of the shell window right below your description. For Exercise 2, type <code>javac -version</code> in the command window. Then paste a picture of the result that you get back into your document. This will tell us whether or not you have the JDK correctly installed and ready to compile your programs. Here's how you'd interpret the results.

folder. (On Unix or the Mac, create this folder inside your personal home directory.) Name your home directory something like cs170home that's easy to remember, and self-explanatory so you can find it when necessary. You can create this on a USB thumb drive or, in the the OCC Computing Center, on your U: drive. Try not to create it inside your "My Documents" folder. You'll find that having spaces in the path sometimes makes things more difficult.

Now, using the command line, create a unit01 folder inside your cs170home folder.

Finally, start the SciTE editor (Sc1.exe) from the command line. If you're using some other editor, start it from the command line instead.

For Exercise 3, describe the steps you followed to complete these three tasks. (If you like, you can just shoot a screenshot of the command window, showing your commands.)



Enter Java Source

- Save empty file inside **unit01** as : **FirstApp.java**
- Type and save (again) the following Java program
 - Note that "println" is short for "print line"; "el-en" not "1-en"

```
// Stephen Gilbert <--- put YOUR name here
// CS 170 - August 30, 2007 <--- Put today's date here

public class FirstApp
{
    public static void main(String[] args)
    {
        System.out.print("Hello, ");
        System.out.print("I love you. ");
        System.out.println("Won't you tell me your name?");
    }
}
```

Slide 9

Enter Java Source

Duration: 00:01:18

Advance mode: Auto

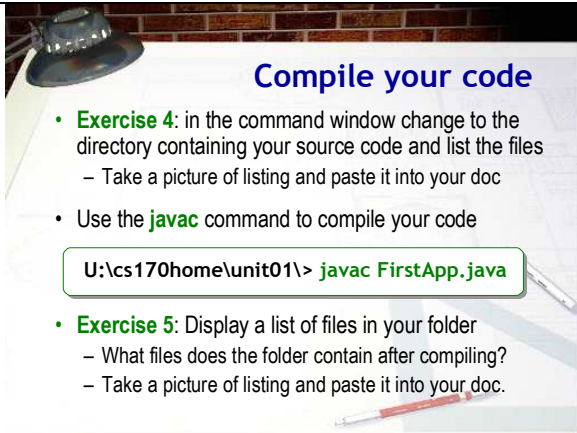
Notes:

When SciTE opens, before you've typed anything at all, save the empty file inside your unit01 folder as FirstApp.java. Java programs are case sensitive, so make sure you capitalize the F i First and the App in App, and leave all the other letters lowercase. Make sure you also don't put any spaces in the file name.

Once you've saved the file, type the following program, and then make sure you save again. (If you try to compile without saving your changes, the compiler won't see your code.) You can tell if you've saved the latest version of the program by looking at the little tab. (see the arrow). If there's an asterisk after the file name that means that your latest changes haven't been saved.

Note that when you type your source code, spelling and punctuation are critical. One thing that might trip you up is the unfamiliar word println which is short for "print line". It ends in an "el-en", not a "1-en". Note also that the Ses in System and String are both capitalized but that most other words are lowercase.

To give you time to type in your code, this slide will pause until you press the next-slide button in the viewer.



Compile your code

- **Exercise 4:** in the command window change to the directory containing your source code and list the files
 - Take a picture of listing and paste it into your doc
- Use the **javac** command to compile your code

```
U:\cs170home\unit01> javac FirstApp.java
```

- **Exercise 5:** Display a list of files in your folder
 - What files does the folder contain after compiling?
 - Take a picture of listing and paste it into your doc.

Slide 10 

Compile your code

Duration: 00:01:24
Advance mode: Auto

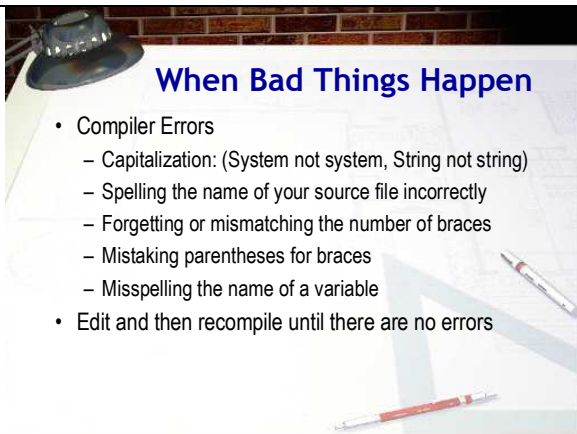
Notes:

Once you've finished editing your source code we need to compile it. We'll do that from a command-shell window. For Exercise 4, change to the directory containing the source code file that you just edited (FirstApp.java) and list all of the files. (There should be only one file in the folder.) Take a picture of your listing and paste it into your exercise document.

Now, use the command `javac FirstApp.java` and press Enter. If all goes well, the compiler will silently convert your Java source code into bytecode. Let's see if it worked.

Display a listing of files in your folder after compiling. You should now have two files, your source code (with the extension .java) and the bytecode file (with the extension .class). For Exercise 5, snap me a picture of your file listing and paste it into your document.

Now, if you don't see two files, or if the compiler printed out a message when you tried to compile your program, it means that you made a syntax error when typing in your code. (Of course, since you were just typing in code that I'd supplied you, in this case that means you made a typing error.) Let's take a look at the different kinds of syntax errors that can occur at this point.



When Bad Things Happen

- Compiler Errors
 - Capitalization: (System not system, String not string)
 - Spelling the name of your source file incorrectly
 - Forgetting or mismatching the number of braces
 - Mistaking parentheses for braces
 - Misspelling the name of a variable
- Edit and then recompile until there are no errors

Slide 11 

When Bad Things Happen

Duration: 00:01:28
Advance mode: Auto

Notes:

There are several kinds of errors you can make at this point that might seem insignificant, but that will prevent your code from compiling. Here are the ones that students encounter most frequently.

First, Java is case sensitive, so capitalization is significant. In this case, both the words `System` and `String` must be capitalized.

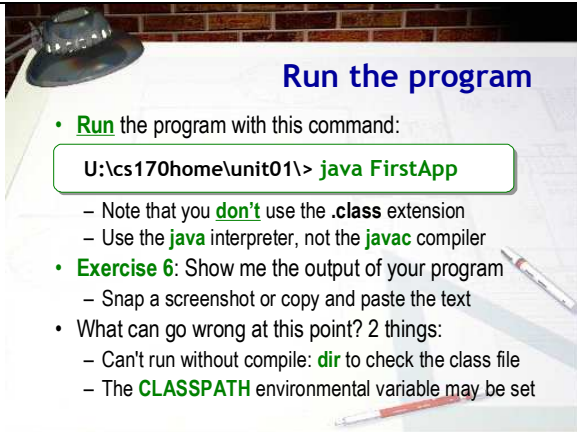
Next, Java is also case sensitive when it comes to file names, even if your operating system (like Windows) is not. Your source file name must match exactly the name that appears after the word `class` when you type in your programming code. Any difference between the two will cause a problem.

Third, Java is very picky about punctuation. All braces, brackets and parentheses must appear in matching pairs, and you cannot substitute curly braces for parentheses or vice-versa.

Finally, when you have variables or methods in your program, it's often easy to misspell the name of a variable or a keyword. When you do this, your program also will fail to compile.

If you had any of these errors (or others), edit and recompile until you have no errors and the compiler produces a .class file. When you've got the class file (and snapped your

screenshot for Exercise 5), click the Next Slide button to continue the lecture.



Run the program

- **Run** the program with this command:
`U:\cs170home\unit01\> java FirstApp`
 - Note that you **don't** use the **.class** extension
 - Use the **java** interpreter, not the **javac** compiler
- **Exercise 6:** Show me the output of your program
 - Snap a screenshot or copy and paste the text
- What can go wrong at this point? 2 things:
 - Can't run without compile: **dir** to check the class file
 - The **CLASSPATH** environmental variable may be set

Slide 12 
Run the program
Duration: 00:01:27
Advance mode: Auto

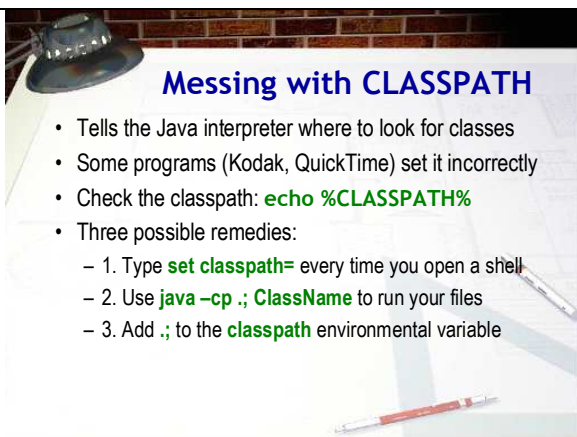
Notes:

Once the compiler has produced a class file, you run the program by launching the JVM with the java command, supplying the name of the class that you want to run. Note that when you compiled the program you had to supply the extension (.java) along with the name of the file. Here, you don't add the class extension; if you do you'll get an error and your program won't run. Also, note that this time we're using the program named java (which launches the interpreter), NOT javac which is the compiler.

After you type this command, shoot me a screenshot of your program running. Also, copy and paste your code into your class exercise document.

If you've got this far, everything should work without problems. If the output isn't as you expect, though, make sure that you've saved and compiled the program before you run it. Students often compile without saving, so that the changes made to the source code don't have any effect. Also, use the dir command to check and make sure that the class file was actually created. If you don't have a class file, you can't run the program.

There's also a second problem that may come up if you're working at home: the CLASSPATH environmental variable may be set incorrectly. Let's take a second to look at that.



Messing with CLASSPATH

- Tells the Java interpreter where to look for classes
- Some programs (Kodak, QuickTime) set it incorrectly
- Check the classpath: `echo %CLASSPATH%`
- Three possible remedies:
 - 1. Type `set classpath=` every time you open a shell
 - 2. Use `java -cp .; ClassName` to run your files
 - 3. Add `.;` to the `classpath` environmental variable

Slide 13 
Messing with CLASSPATH
Duration: 00:01:42
Advance mode: Auto

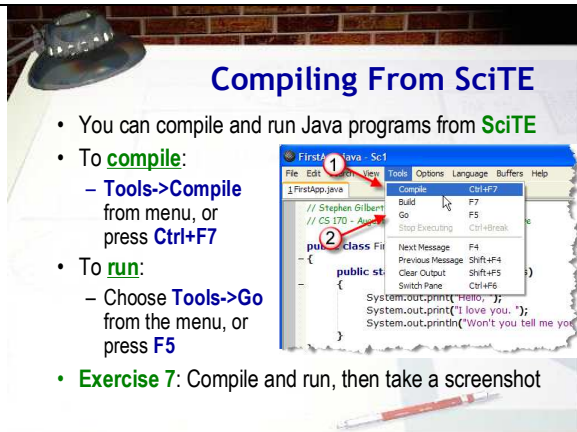
Notes:

The CLASSPATH is an environmental variable kind of like the PATH variable. The PATH variable is used by the shell to find programs you want to run, while the CLASSPATH is used by the Java interpreter (and the compiler) to find Java library classes when your programs run. If you get an error message when you try to run your program, it may be that another program has set the CLASSPATH environmental variable. (Quicktime for Java does this, as well as some Oracle tools and Kodak digital cameras.)

You can find out by typing: `echo %CLASSPATH%` at the command prompt. If the system responds with anything other than `%CLASSPATH%`, then the classpath has been set and you'll need to "unset it" by typing `SET CLASSPATH=` at the command prompt and then trying to run your program again.

As an alternative, you can use the `cp` switch when invoking the Java interpreter. Just use `java -cp ./ FirstApp` to include the "current directory" (the dot, followed by the forward slash) in the program's classpath.

Finally, for a permanent fix, locate the CLASSPATH in the system environmental variables (just like you did for the PATH), and add the current directory (which is a `.`; on Windows and a `.` on Unix and the Mac) to the existing CLASSPATH values.



Compiling From SciTE

- You can compile and run Java programs from **SciTE**
- To **compile**:
 - Tools->Compile** from menu, or press **Ctrl+F7**
- To **run**:
 - Choose **Tools->Go** from the menu, or press **F5**
- Exercise 7**: Compile and run, then take a screenshot

Slide 14

Compiling From SciTE

Duration: 00:01:05

Advance mode: Auto

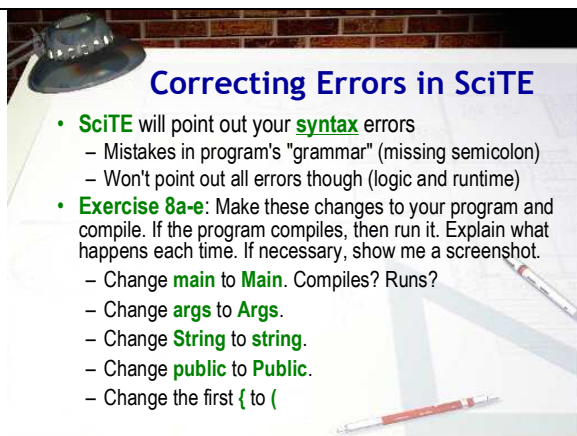
Notes:

If you're using the Scintilla Text Editor (see the links page for the download), you can actually compile and run your Java programs from within the editor. SciTE also allows you to click on error messages and "jump" to the offending line of code.

You can compile your programs from within SciTE by choosing **Tools->Compile** from the menu, or pressing **Ctrl+F7**. Note that this just issues exactly the same shell commands that you typed manually. If you can't compile from the command-line, then you won't be able to compile from within SciTE.

To run your program, choose **Tools->Go** from the menu, or press **F5**. The output appears in a separate output pane. If there are any errors, they'll appear there as well.

For Exercise 7, compile and run `FirstApp.java` from within the SciTE editor and then shoot me a screenshot.



Correcting Errors in SciTE

- SciTE** will point out your **syntax** errors
 - Mistakes in program's "grammar" (missing semicolon)
 - Won't point out all errors though (logic and runtime)
- Exercise 8a-e**: Make these changes to your program and compile. If the program compiles, then run it. Explain what happens each time. If necessary, show me a screenshot.
 - Change **main** to **Main**. Compiles? Runs?
 - Change **args** to **Args**.
 - Change **String** to **string**.
 - Change **public** to **Public**.
 - Change the first `{` to `(`

Slide 15

Correcting Errors in SciTE

Duration: 00:01:60

Advance mode: Auto

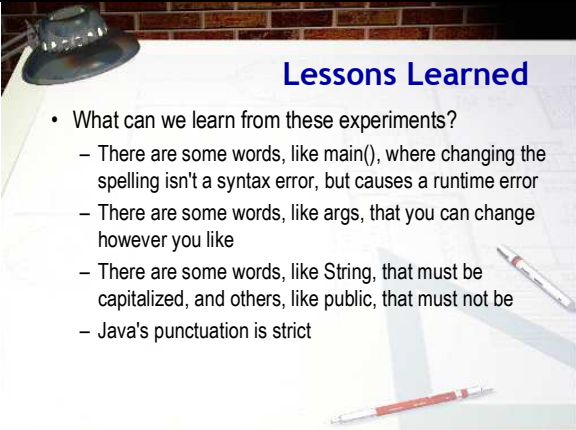
Notes:

When you make a syntax error (that is, an error in the Java grammar that prevents your code from compiling), SciTE will point out your error by highlighting the line with a little yellow ball in the left margin, when you double-click the error message in the output window. This makes it a little easier to get your code compiled correctly; of course it doesn't help at all with runtime or logic errors.

Let's see how that works by making some changes to `FirstApp.java` and seeing what results you come up with. For each of these changes in Exercise 8a-e, first change your source code and save your file. Then, try to compile. If the program compiles, then try to run it. After each experiment, return the file to its original condition before you make the next set of changes. If an error occurs, shoot a screenshot. If everything goes OK, just say so.

a) Change the word `main` to `Main` (with a capital M). Save your file and compile. Do you get an error? If not, then run the program. Do you get an error then? If so, what does the error message say?

b) Change the word `args` to `Args`. Does it compile? Does it

	run? c) Change String to lowercase string d) Change lowercase public to capitalized Public e) Change the first left brace { to a left parentheses. After you're finished, let's talk about what those things mean.
 Slide 16 🎧 Lessons Learned Duration: 00:01:32 Advance mode: Auto • What can we learn from these experiments? – There are some words, like <code>main()</code>, where changing the spelling isn't a syntax error, but causes a runtime error – There are some words, like <code>args</code>, that you can change however you like – There are some words, like <code>String</code>, that must be capitalized, and others, like <code>public</code>, that must not be – Java's punctuation is strict	Notes: So, what can we learn from these experiments? First, changing some words, like <code>main()</code>, doesn't stop your code from compiling. Naming a method capital <code>Main</code> instead of lowercase <code>main</code> isn't a syntax error, but causes a different kind of error: a runtime error. Second, some of the words we'll use can be changed however you like, and it won't affect the program at all, like the name <code>args</code>. You can make it uppercase, lowercase, or even use a different word altogether, like <code>bob</code>, and Java won't care. Third, some words must be in uppercase, like <code>String</code>, while others, like <code>public</code>, must be in lowercase. Those that must be in lowercase are the keywords or the build-in vocabulary of the Java language. Those that must be in uppercase are the predefined classes in the Java class library. Java really doesn't care if you use a lowercase <code>s</code> for <code>string</code>, but it won't find the predefined <code>String</code> class if you do. Finally, Java's punctuation is extremely strict. You can't confuse braces, brackets or parentheses: each must be used in a particular place and has a particular meaning. The same goes for colons and semicolons, commas, single and double quotes. But that's kind of the topic of the next lecture, isn't it? What are the rules of the Java language, and what does that code you just typed in actually mean? We'll look at that next.